

DocuControl — OWASP-Sicherheitsbericht: Pi-Host-Ebene

Gerät: docucontrol3 / Pi5_Display (192.168.0.218 / .11), Raspberry Pi 5, Debian Trixie

Prüfmethode: Live-Systemaudit über SSH (OS, Netzwerk, SSH, sudo, Docker-Konfiguration, Updates) **Datum:** 22.06.2026 **Ergänzt:** den bereits vorliegenden Bericht

`docucontrol_owasp_sicherheitsbericht.md` (Web-App-Ebene, Flask/Docker-Container-Innenleben). Dieser Bericht betrachtet zusätzlich den **Host** selbst — also die Ebene unterhalb der Web-Anwendung.

Zusammenfassung

Während der Web-App-Layer (Flask/Docker-Innenleben) bereits gehärtet wurde, zeigt das Host-Level-Audit zusätzliche Punkte, die **bewusste Entscheidungen** erfordern, bevor sie behoben werden — anders als die bisherigen Fixes sind diese teils mit Funktionsverlust oder Zugriffsänderungen über die ganze Geräteflotte hinweg verbunden. Es wurde daher **nichts automatisch verändert**, dieser Bericht dient als Entscheidungsgrundlage.

1. Kritischer Befund

1.1 Gemeinsames Klartext-Passwort, im Git-Repository dokumentiert

- Das SSH/sudo-Passwort `Xtend1478` ist identisch auf **allen** Fleet-Geräten (DocuPi-3000, DocuControl .171, Pi5_Display/docucontrol3 .218) im Einsatz.
- Das Passwort steht **im Klartext** in `CLAUDE.md`, einer Datei, die im Repository versioniert und committed ist (mehrfach referenziert, u. a. Zeilen 109/110/122/412/460).
- **Risiko:** Wer Lese-Zugriff auf das Repository (oder dessen Git-Historie) hat, kennt automatisch das Root-Äquivalent-Passwort für die gesamte Geräteflotte. Ein einzelnes kompromittiertes Repo-Zugriffsrecht (z. B. ein versehentlich öffentlich gemachtes Repo, ein abgelaufener Collaborator-Zugang) gefährdet sofort **alle** Geräte gleichzeitig.
- **OWASP-Kategorie:** A02 Cryptographic Failures / A05 Security Misconfiguration

Empfehlung (nicht automatisch umgesetzt — siehe Begründung unten): Pro Gerät ein individuelles, langes Passwort (oder besser: SSH-Key-only-Login, sudo ganz ohne Passwort-Fallback) und das Klartext-Passwort aus `CLAUDE.md` /Git-Historie entfernen (`git filter-repo` o. ä., falls die Historie bereinigt werden soll).

2. Hohe Befunde

2.1 Docker-Container läuft `privileged: true` + `network_mode: host`

```
services:
  docucontrol:
    network_mode: host
    privileged: true
    volumes:
      - /sys:/sys:ro
      - /dev:/dev
      ...
```

- Der Container hat damit **vollen Zugriff auf alle Host-Geräte** (`/dev`), **das komplette Host-Netzwerk-Stack** (kein Netzwerk-Namespaces-Isolation) und praktisch **root-äquivalente Rechte** auf dem Host (CAP_SYS_ADMIN u. a. via `privileged`).
- Diese Rechte sind funktional begründet (USB-Drucker via CUPS, `nmcli` für Netzwerkverwaltung, USB-Stick-Mount, Watchdog-HAT) — eine Web-App mit so weitreichenden Container-Rechten bedeutet aber: **jede Schwachstelle in der Flask-App (z. B. eine noch unentdeckte RCE) führt direkt zur vollständigen Host-Kompromittierung**, nicht nur zur Kompromittierung eines isolierten Containers.
- **OWASP-Kategorie:** A05 Security Misconfiguration

Einschätzung: Eine Einschränkung (z. B. gezielte `--device` / `--cap-add` statt `privileged: true` , Bridge-Netzwerk statt `host`) würde die Angriffsfläche deutlich reduzieren, erfordert aber eine sorgfältige Einzelanalyse, welche Capability für welche Funktion (CUPS, `nmcli`, USB-Mount, Watchdog-HAT) tatsächlich nötig ist — Risiko von Funktionsausfällen bei zu aggressiver Einschränkung. **Nicht ohne Test in einem Wartungsfenster empfehlenswert.**

3. Mittlere Befunde

3.1 SSH erlaubt Passwort-Authentifizierung

- `sshd -T` zeigt `passwordauthentication yes` — SSH-Login ist nicht auf Public-Key beschränkt, obwohl ein Key (`docucontrol_id`) bereits im Einsatz ist.
- Kombiniert mit Befund 1.1 (schwaches/geteiltes Passwort) ist das ein klassischer Brute-Force-Angriffsvektor, **sofern** das Gerät jemals über das LAN hinaus erreichbar würde (aktuell durch Netzsegmentierung gemindert).
- **OWASP-Kategorie:** A07 Identification and Authentication Failures

Empfehlung: `PasswordAuthentication no` setzen, sobald sichergestellt ist, dass der SSH-Key-Zugang von allen benötigten Arbeitsplätzen aus funktioniert (sonst Aussperr-Risiko).

3.2 rpcbind (Port 111) auf allen Interfaces erreichbar

- `rpcbind` lauscht auf `0.0.0.0:111` (TCP+UDP) — Teil des NFS/RPC-Stacks.
- Auf diesem Gerät wird kein NFS verwendet (Netzwerk-Speicherort läuft über SMB/CIFS, nicht NFS) — der Dienst ist vermutlich eine ungenutzte Paketabhängigkeit.
- Unnötig erreichbare Dienste vergrößern die Angriffsfläche ohne Funktionsnutzen.
- **OWASP-Kategorie:** A05 Security Misconfiguration

Empfehlung: Prüfen, welches Paket `rpcbind` zieht (`apt-cache rdepends rpcbind`), und bei Nichtbedarf `systemctl disable --now rpcbind` + ggf. Paket entfernen.

3.3 Kein fail2ban/sshguard installiert

- Keine automatische Sperrung bei wiederholten SSH-Fehlversuchen auf Host-Ebene (die App-Ebene hat seit dem letzten Review einen eigenen Lockout-Mechanismus, das betrifft aber nur den Web-Login, nicht SSH).
- Aktuell unkritisch: 0 fehlgeschlagene SSH-Versuche in den letzten 30 Tagen (reiner LAN-Betrieb, keine Internet-Exposition).
- **OWASP-Kategorie:** A07 Identification and Authentication Failures

Empfehlung: Optional `fail2ban` mit SSH-Jail nachrüsten, geringer Aufwand, kein Funktionsrisiko.

4. Positive Befunde

- Nur **ein** menschlicher Account (`docucontrol`) mit Login-Shell — kein Wildwuchs an Benutzerkonten.
 - `PermitRootLogin without-password` — Root-Login nur per Key, nicht per Passwort.
 - `PermitEmptyPasswords no`.
 - CUPS (Port 631) ist korrekt nur auf `127.0.0.1 / ::1` gebunden, nicht netzwerkweit erreichbar.
 - System ist aktuell — nur ein einziges (nicht sicherheitskritisches) Paket-Update offen (`gststreamer1.0-plugins-good` Security-Update, unkritisch für diesen Anwendungsfall).
 - 0 fehlgeschlagene SSH-Login-Versuche in 30 Tagen — kein Hinweis auf aktive Angriffsversuche.
 - `nftables`-Portweiterleitung (`80→5000`, `443→5443`) korrekt interface-basiert konfiguriert.
-

5. Offene Ports (Übersicht)

Port	Dienst	Bindung	Bewertung
22	SSH	0.0.0.0	OK, Passwort-Auth siehe 3.1
111	rpcbind	0.0.0.0	unnötig, siehe 3.2
631	CUPS	127.0.0.1:::1	OK, lokal gebunden
5000	Flask HTTP (Kiosk)	0.0.0.0	OK, Access-Control bereits gehärtet
5443	Flask HTTPS	0.0.0.0	OK, neu eingerichtet (selbstsigniert)
9100	TCP-Druckempfang	0.0.0.0	bewusst ohne Auth, siehe vorheriger Bericht
34211	(Docker-internes Loopback)	127.0.0.1	unkritisch, nicht extern erreichbar

6. Empfohlene Priorisierung

Priorität	Maßnahme	Risiko bei Umsetzung
1	Klartext-Passwort aus <code>CLAUDE.md</code> /Git-Historie entfernen, individuelle Passwörter je Gerät	gering, nur Dokumentationspflege
2	rpcbind deaktivieren, falls ungenutzt	gering, kurzer Test nach Deaktivierung empfohlen
3	<code>PasswordAuthentication no</code> für SSH	mittel — vorher Key-Zugang von allen Arbeitsplätzen verifizieren
4	fail2ban für SSH nachrüsten	gering
5	Docker-Capabilities von <code>privileged: true</code> auf gezielte Capabilities reduzieren	hoch — sorgfältige Einzelanalyse + Testfenster nötig, Funktionsausfall-Risiko

Fazit

Der bisherige Fokus lag auf der Web-Anwendung (Flask/Docker-Innenleben) und wurde erfolgreich gehärtet. Das Host-Level-Audit zeigt, dass das **größte verbleibende Risiko organisatorisch** ist (geteiltes Klartext-Passwort in der Dokumentation), nicht technisch. Die technischen Punkte (rpcbind, SSH-Passwort-Auth, Docker-Privilegien) sind in der aktuellen LAN-only-Betriebsumgebung von

begrenzter praktischer Relevanz, sollten aber bei einer Erweiterung des Netzwerkzugriffs (z. B. VPN-Fernzugriff, weitere Standorte) vorher angegangen werden.

Dieser Bericht wurde im Rahmen der laufenden Entwicklungsarbeit am DocuControl-Projekt erstellt und spiegelt den Stand vom 22.06.2026 wider. Es wurden bewusst keine automatischen Änderungen am Host vorgenommen — alle Empfehlungen erfordern eine explizite Entscheidung des Betreibers.